

PROJECT REPORT

TaskFlow: End-to-End DevOps Implementation and Automation

A progressive, hands-on implementation covering application development, containerization, CI/CD, configuration management, infrastructure as code, container orchestration, monitoring, and security

This report documents the design, implementation, and evaluation of TaskFlow – a REST API used as a single, coherent vehicle to build and connect the major categories of modern DevOps tooling, from source control through to production-style observability and security scanning.

Abstract

Modern software delivery depends on more than writing application code. Teams require repeatable processes for building, testing, configuring, deploying, monitoring, and securing services. Manual approaches to these activities introduce inconsistency, delay, and operational risk. DevOps practices address this gap by integrating development and operations through automation, infrastructure as code, and continuous feedback.

This report documents TaskFlow, an end-to-end DevOps implementation in which a FastAPI-based task-management REST API was taken from a local Linux environment through containerization, continuous integration, configuration management, infrastructure as code, cloud simulation and real cloud practice, Kubernetes deployment, Helm packaging, monitoring, and basic security and network analysis.

The development environment was established on an Ubuntu Server virtual machine accessed via SSH, with remote editing through VS Code Remote-SSH. The application was version-controlled with Git and GitHub, persisted with SQLite and SQLAlchemy, tested with pytest, and containerized with Docker. Continuous integration was implemented with both GitHub Actions and a self-hosted Jenkins instance. Server configuration was automated with Ansible. Infrastructure was defined with Terraform against LocalStack and a real AWS free-tier account. The application was deployed to a Minikube Kubernetes cluster with Deployments, Services, ConfigMaps, and Secrets, then packaged with Helm. Observability was added with Prometheus metrics and Grafana dashboards. Network traffic was inspected with tshark/Wireshark, and container images were scanned with Trivy.

The project deliberately used free and local tooling wherever possible, and is explicit about its scope: the environment is portfolio-scale rather than production-scale; Minikube was used instead of a managed Kubernetes control plane; SQLite was used instead of a managed database; and continuous deployment was demonstrated within the limits of a local cluster. The outcome is a coherent, technically documented demonstration of how the major DevOps tool categories connect in practice, along with a detailed record of real troubleshooting and lessons learned.

Keywords: DevOps, Continuous Integration, Continuous Deployment, Docker, Kubernetes, Jenkins, Ansible, Terraform, LocalStack, AWS, Helm, Prometheus, Grafana, FastAPI, Infrastructure as Code

Table of Contents

- 1. Introduction
- 2. Technology and Practice Review
- 3. Requirement Specifications
- 4. System Design
- 5. DevOps Automation Strategy
- 6. System Development Methodology
- 7. System Testing and Evaluation
- 8. Results and Discussion
- 9. Conclusion and Future Work
- References
- Appendices

1. Introduction

1.1 Background

Software systems are no longer delivered as one-time releases installed on a single machine. They are built continuously, tested automatically, deployed to multiple environments, scaled under load, and observed in production. The practices that enable this model are collectively known as DevOps a cultural and technical approach that shortens the path from change to reliable delivery by automating the work that used to sit between development and operations.

Manual deployment processes create well-known problems: environment drift ("it works on my machine"), slow feedback when tests run only at the end of a cycle, configuration that exists only in an engineer's memory, and limited visibility into how a service behaves after release. DevOps counters these problems with concrete techniques:

- Continuous Integration (CI) every code change is automatically built and tested
- Continuous Delivery / Deployment (CD) validated changes can be released with minimal manual steps
- Containerization applications and dependencies are packaged into portable images
- Infrastructure as Code (IaC) infrastructure is declared in version-controlled files rather than clicked together in a console
- Configuration Management server state is described and applied repeatedly, idempotently
- Orchestration containerized services are scheduled, healed, and scaled by a cluster manager
- Monitoring and Observability metrics and logs make system behaviour visible
- Security integration (DevSecOps) scanning and network awareness are part of the pipeline, not an afterthought

This project uses a single application TaskFlow, a small task-management REST API as the vehicle through which these practices are learned and connected. Rather than treating tools in isolation, the work advances layer by layer so each tool has a clear place in one coherent pipeline.

1.2 Problem Statement

Writing an application that runs on a laptop is a different problem from moving that application reliably through version control, automated testing, container packaging, multi-environment configuration, cloud infrastructure, cluster deployment, monitoring, and basic security checks. Each of those steps involves tools, conventions, and failure modes that are hard to learn from disconnected tutorials.

The practical problem addressed by this project is: how can a single, realistic application be used to build and demonstrate an end-to-end DevOps workflow using free and local tooling, while documenting real errors, fixes, and limitations honestly?

TaskFlow was chosen because it is small enough to implement and reason about, yet large enough to require persistence, testing, containerization, and external configuration the same categories of concern that appear in production services.

1.3 Objectives

ID	Objective
O1	Build a functional FastAPI REST API (TaskFlow) with CRUD operations and persistent storage
O2	Establish a Linux server development environment accessible via SSH and remote editing
O3	Use Git and GitHub for version control with SSH-based authentication
O4	Containerize the application with Docker and verify it runs outside the host Python environment
O5	Implement automated continuous integration using GitHub Actions and Jenkins

O6	Automate server configuration with Ansible (inventory, playbook, idempotency)
O7	Provision infrastructure with Terraform against LocalStack and real AWS free-tier resources
O8	Deploy the application to Kubernetes (Minikube) with Deployments, Services, ConfigMaps, and Secrets
O9	Package Kubernetes resources with Helm
O10	Instrument the application for Prometheus metrics and visualize them in Grafana
O11	Inspect network traffic with tshark/Wireshark and scan container images with Trivy
O12	Document troubleshooting, limitations, and lessons learned

1.4 Scope

The scope is deliberately bounded to what can be implemented and verified on personal hardware without paid cloud subscriptions beyond optional free-tier usage.

Implemented and tested:

- Ubuntu Server VM, SSH, VS Code Remote-SSH, Git/GitHub
- FastAPI TaskFlow API with SQLite/SQLAlchemy and pytest
- Docker image and container
- GitHub Actions CI and Jenkins CI pipelines
- Ansible playbook against a target VM
- Terraform for S3 on LocalStack and on real AWS (create/verify/destroy)
- Minikube Deployment (2 replicas), Service, ConfigMap, Secret
- Helm chart install
- Prometheus scraping and Grafana dashboard panels
- tshark capture analysis and Trivy image scan

Explored or optional:

- AWS free-tier practice limited to carefully scoped, destroyed resources
- TLS/HTTPS termination discussed and partially explored, not fully production-hardened
- Non-root container hardening discussed; full production policy not claimed

Out of scope / future work:

- Managed Kubernetes (EKS/GKE/AKS) a managed control plane is not free
- Production database (e.g., managed PostgreSQL)
- Full GitOps (e.g., Argo CD), centralized logging, advanced alerting
- Load testing, multi-region high availability, autoscaling policies at scale

1.5 Outcomes

By the end of this project, the following competencies were demonstrated: operating a headless Linux server over SSH; structuring a Python service with tests and containers; writing CI pipelines in both managed (GitHub Actions) and self-hosted (Jenkins) forms; expressing server desired-state with Ansible; declaring cloud resources with Terraform; deploying and configuring applications on Kubernetes; packaging releases with Helm; exposing and graphing application metrics; and reasoning about network exposure and image vulnerabilities using concrete evidence from packet captures and scans.

2. Technology and Practice Review

2.1 DevOps Overview

DevOps is not a single tool. It is a set of practices that reduce the delay and friction between writing software and running it reliably—combining culture (shared ownership of production outcomes), automation (repeatable pipelines), measurement (metrics and logs), and version-controlled configuration and documentation.

2.2 Continuous Integration and Delivery

Continuous Integration means developers merge small changes frequently and each merge triggers an automated build and test process. Continuous Delivery extends this by keeping software in a releasable state; Continuous Deployment goes further by releasing validated changes automatically. In this project, CI was implemented fully; CD was demonstrated in the sense of deploying to a local Kubernetes cluster, but not as a fully automated production release train.

2.3 Containerization

Containers package an application with its runtime dependencies so the same artifact runs consistently on a developer machine, in CI, and on a cluster node. Docker was used to build and run the TaskFlow image—the foundation on which Kubernetes scheduling and Helm packaging later depend.

2.4 Infrastructure as Code

Infrastructure as Code treats servers, networks, and cloud resources as declarative definitions stored in version control. Terraform was selected because it is widely used, supports many providers, and allows the same configuration patterns to target both LocalStack (local AWS API simulation) and real AWS.

2.5 Configuration Management

Configuration management tools apply a desired state to machines—packages installed, services running, users present, firewall rules enabled—and are designed to be idempotent: running the same playbook twice should not make unnecessary changes. Ansible was used because it is agentless over SSH, matching the remote-server workflow already established.

2.6 Container Orchestration

Kubernetes schedules containers across nodes, keeps replica counts healthy, and provides service discovery and load balancing. Minikube provided a single-node cluster suitable for learning without the cost of a managed control plane. Core objects used included Deployment, ReplicaSet, Pod, Service, ConfigMap, and Secret.

2.7 Monitoring and Observability

Monitoring answers whether the system is healthy; observability supports asking new questions about behaviour. Prometheus scrapes time-series metrics; Grafana visualizes them. TaskFlow was instrumented to expose metrics, Prometheus was configured to scrape the Service, and Grafana dashboards displayed request rate and approximate latency percentiles.

2.8 DevSecOps and Network Analysis

Security practices were introduced at a practical, evidence-based level: packet capture with tshark showed that HTTP payloads on the lab network were readable in plaintext, motivating TLS in real deployments; Trivy scanned the Docker image for known vulnerabilities in base OS packages and application dependencies. These activities demonstrate evidence-based security awareness rather than a claim of a complete security program.

2.9 Tools Used

Tool	Category	Role in TaskFlow
Git / GitHub	Version control	Source history, remote collaboration, CI triggers
FastAPI / Uvicorn	Application	REST API implementation and ASGI server
SQLAlchemy / SQLite	Data	ORM and file-based persistence
pytest	Testing	Automated API tests for CI
Docker	Containers	Image build and runtime isolation
GitHub Actions	CI (managed)	On-push test and image build
Jenkins	CI (self-hosted)	Pipeline equivalent running in Docker on the VM
Ansible	Config management	Idempotent setup of target Ubuntu VM
Terraform	IaC	S3 bucket lifecycle on LocalStack and AWS
LocalStack	Cloud simulation	Local AWS API endpoints for safe practice
AWS (free tier)	Cloud	Real S3 create/destroy with budget safeguards
Minikube / kubectl	Orchestration	Local Kubernetes cluster and control
Helm	Packaging	Templated Kubernetes release
Prometheus	Metrics	Scraping TaskFlow /metrics
Grafana	Visualization	Dashboards for rate and latency
tshark / Wireshark	Network analysis	Packet capture of HTTP traffic
Trivy	Security scanning	Container image vulnerability report

3. Requirement Specifications

3.1 Development Environment

The primary development environment was chosen to mirror how operators interact with real servers: a headless Linux system reached over SSH, not a local GUI-first workflow.

Component	Choice	Rationale
Host OS	Windows	Existing workstation
Hypervisor	VMware Workstation Player	Free for personal use; stable Ubuntu Server guests
Guest OS	Ubuntu Server (LTS)	Headless, matches cloud instance shape; lighter than Desktop
Remote access	SSH from Windows Terminal/PowerShell	Industry-standard remote administration
Editor	VS Code + Remote-SSH	Full IDE experience while files remain on Linux
Alternative considered	WSL2 (not chosen)	VM preferred for realistic server isolation and later multi-VM Ansible work

3.2 Hardware Requirements

The project ran on a personal laptop capable of hosting a VM with approximately 2 CPU cores, 4 GB RAM (adjustable), and roughly 25 GB disk for the guest, later expanded when disk pressure appeared during LocalStack/Docker/Minikube usage. The key requirement is enough headroom for Docker, Minikube, Jenkins, and monitoring components concurrently or in sequence.

3.3 Software Requirements

Software	Purpose
Ubuntu Server	Primary runtime and learning OS
Python 3, pip, venv, build-essential	Application runtime and package builds
Git	Version control
FastAPI, Uvicorn, SQLAlchemy, pytest	API, server, ORM, tests
Docker	Container build and run
Jenkins (in Docker)	Self-hosted CI
Ansible	Configuration management
Terraform	Infrastructure as Code
LocalStack	Local AWS API simulation
AWS CLI / account (free tier)	Real cloud practice with safeguards
Minikube, kubectl	Local Kubernetes

Helm	Kubernetes packaging
Prometheus, Grafana	Metrics and dashboards
tshark / Wireshark	Packet capture and analysis
Trivy	Image vulnerability scanning
VS Code + Remote-SSH	Remote development

3.4 Functional Requirements

ID	Requirement
FR1	The system shall expose REST endpoints for health and task CRUD operations
FR2	Task data shall persist across application restarts (SQLite)
FR3	Automated tests shall verify core API behaviour (pytest)
FR4	The application shall run inside a Docker container from a Dockerfile
FR5	A CI pipeline shall run tests and build the image on code push (GitHub Actions)
FR6	An equivalent pipeline shall run in Jenkins on the development VM
FR7	Ansible shall configure a target Ubuntu host (packages, user, firewall) idempotently
FR8	Terraform shall create and destroy an S3 bucket on LocalStack and on AWS
FR9	Kubernetes shall run multiple TaskFlow replicas behind a Service
FR10	ConfigMaps and Secrets shall inject configuration into pods
FR11	Helm shall package and install the application as a release
FR12	Prometheus shall scrape application metrics; Grafana shall display selected graphs
FR13	Network capture and image scanning shall produce analyzable artifacts

3.5 Non-Functional Requirements

ID	Category	Requirement
NFR1	Reproducibility	Environment setup and pipelines shall be documented and scriptable
NFR2	Portability	The container image shall run independently of the host Python install
NFR3	Maintainability	Configuration shall live in version control (playbooks, Terraform, manifests, charts)
NFR4	Observability	Key request metrics shall be exposable and graphable
NFR5	Security awareness	Unencrypted traffic and image CVEs shall be demonstrable with tools
NFR6	Cost control	Cloud experiments shall use free tier / budgets and destroy resources after use
NFR7	Clarity	Limitations and optional work shall be stated explicitly in documentation

3.6 Cost Considerations

No paid DevOps SaaS subscriptions were required. Tooling was open-source or free for personal use (VMware Workstation Player, Ubuntu, Docker, Jenkins, Ansible, Terraform, Minikube, Prometheus, Grafana, Trivy). AWS usage was constrained to free-tier-compatible experiments with a low budget alert and explicit destroy steps; the Kubernetes phase avoided EKS because a managed control plane is charged continuously. LocalStack avoided cloud cost for iterative Terraform practice. Monetary spend was intended to remain at or near zero when cleanup habits were followed.

4. System Design

4.1 Overall Architecture

The architecture connects a developer workstation to a Linux VM, a GitHub repository, CI systems, container images, configuration and infrastructure automation, a local Kubernetes cluster, and monitoring:

Developer (VS Code Remote-SSH) → GitHub → CI (GitHub Actions / Jenkins) → Docker Image → Ansible (config) / Terraform (LocalStack + AWS) → Minikube Cluster (Deployment, Service, ConfigMap, Secret) → Prometheus → Grafana

Reading this end to end: code is written on the VM via Remote-SSH, pushed to GitHub, validated by CI, packaged as a Docker image, optionally configured onto hosts with Ansible, supported by Terraform-managed object storage practice, deployed into Minikube, and observed through Prometheus and Grafana.

4.2 Development Environment

Windows hosts VMware, which runs Ubuntu Server. Daily work never depends on the VM console: SSH provides a terminal, and VS Code Remote-SSH mounts the remote filesystem for editing. This mirrors the same muscle memory used against real cloud instances – authenticate, work remotely, leave the server headless.

4.3 Application Architecture

TaskFlow is a FastAPI application exposing endpoints such as GET /health, GET /, GET/POST /tasks, GET/PUT/DELETE /tasks/{id}. Pydantic models validate payloads. SQLAlchemy maps Task entities to a SQLite database file so data survives process restarts – an early in-memory implementation was replaced once realistic persistence was required.

4.4 Version Control Architecture

A local Git repository on the VM tracks application code, tests, Docker metadata, CI workflows, Ansible content, Terraform roots, Kubernetes manifests, Helm charts, and documentation. GitHub hosts the remote. Authentication from the VM to GitHub uses a dedicated Ed25519 SSH key pair, separate from the Windows-to-VM SSH login credentials.

4.5 Docker Architecture

A Dockerfile installs dependencies from requirements.txt, copies the application package, exposes port 8000, and starts Uvicorn. The image is built and run on the VM; later, the same image concept is built inside Minikube's own Docker environment for cluster scheduling. Operational issues encountered included container name conflicts on re-run and Docker-socket permission concerns when CI tools needed to build images.

4.6 Continuous Integration Architecture

GitHub Actions runs on GitHub-hosted runners on push. A workflow checks out the repository, sets up Python, installs dependencies, runs pytest, and builds the Docker image – providing fast feedback without operating CI infrastructure.

4.7 Jenkins Architecture

Jenkins runs as a Docker container on the VM, offering a self-hosted contrast to GitHub Actions. A Jenkinsfile encodes stages analogous to the GitHub workflow. Practical issues included missing language runtimes inside the Jenkins environment, package conflicts, and the need for a Docker CLI inside the Jenkins container in addition to socket access to the host Docker daemon – all diagnosed and fixed as part of the implementation.

4.8 Ansible Architecture

The original Ubuntu VM acts as the Ansible control node. A second, cloned Ubuntu VM acts as the target. Passwordless SSH using a dedicated key connects them. An inventory file lists the target; a playbook declares tasks such as updating the apt cache, installing Docker, ensuring a deployer user and group membership, and enabling firewall rules. Two consecutive runs demonstrated idempotency: the first reported multiple changes, the second reported minimal or no meaningful drift.

4.9 Terraform Architecture

Terraform configurations define an S3 bucket. Against LocalStack, the AWS provider is pointed at local endpoints; a path-style addressing fix was required for reliable bucket operations. The full lifecycle `apply`, `verify`, `destroy`, `re-apply` was practiced. Against real AWS, a dedicated IAM user, a budget alert, and a strict destroy discipline limited cost risk. State files and provider lock files were gitignored appropriately.

4.10 Kubernetes Architecture

Minikube provides a single-node cluster using the Docker driver to avoid nested virtualization issues. A Deployment requests two replicas of the TaskFlow image. A Service of type NodePort publishes the application on a high port on the node IP. Deleting a pod demonstrated self-healing as the ReplicaSet created a replacement. ConfigMaps supplied non-sensitive settings; Secrets supplied sensitive-style values for learning injection patterns. Image visibility required building inside Minikube's own Docker context so the cluster could pull the image without an external registry.

4.11 Helm Architecture

A Helm chart templated Deployment, Service, and ConfigMap resources using `values.yaml`. `helm template` verified rendering; `helm install` created a release. Raw manifests were retained in the repository to show progression from handwritten YAML to a packaged chart.

4.12 Monitoring Architecture

The application exposes a metrics endpoint consumed by Prometheus. Service annotations enable discovery. Grafana connects to Prometheus as a data source and presents panels such as request rate and a latency percentile query. Resource limits were considered because the VM simultaneously hosts multiple stack components.

4.13 Networking and Security Architecture

Client requests reach TaskFlow over HTTP on the lab network. Captures taken with `tshark` and inspected in Wireshark showed request and response bodies in plaintext, illustrating why TLS is required beyond a learning cluster. Trivy scanned the application image, reporting OS and dependency findings; actionable application-level dependency updates were identified. Full production TLS termination at an Ingress was discussed as the standard pattern but not claimed as a completed hardened deployment.

5. DevOps Automation Strategy

5.1 Lifecycle

The project follows the familiar loop: plan, code, build, test, release, deploy, operate, monitor, and improve. Each phase of implementation mapped onto one or more stages of this loop rather than treating tools as an unordered checklist.

5.2 Automation Strategy

Area	Manual baseline	Automated approach in TaskFlow
Testing	Manual Swagger clicks	pytest in local runs and CI
Build	Ad-hoc docker build	CI builds image on push
Server setup	apt commands by hand	Ansible playbook on target VM
Cloud resources	Console clicking	Terraform apply/destroy
App placement	docker run on one host	Kubernetes Deployment + Service
Release packaging	Copy YAML by hand	Helm chart values and templates
Health visibility	curl occasionally	Prometheus scrape + Grafana panels

5.3 CI/CD Strategy

CI was prioritized first because automated tests give later stages something meaningful to gate on. Two CI implementations were built intentionally: GitHub Actions for a managed experience, and Jenkins for a self-hosted experience common in enterprise environments. CD in the strong sense (fully automatic promotion to production) was not claimed; deployment to Minikube and Helm releases demonstrate deployment automation building blocks.

5.4 Infrastructure as Code Strategy

Terraform was introduced only after application and CI basics were stable. LocalStack allowed rapid iteration without cloud risk; real AWS exercises taught account hygiene – dedicated IAM users, budget alerts, and destroy discipline.

5.5 Configuration Management Strategy

Ansible encoded the kinds of steps previously performed manually. Idempotency was treated as a first-class success criterion, verified by consecutive playbook runs.

5.6 Containerization and Orchestration Strategy

Docker provided a portable unit of deployment. Kubernetes added desired-state replication and service abstraction. Helm reduced duplication when the same objects needed consistent configuration across installs.

5.7 Monitoring and Security Strategy

Metrics were added after the service was reachable on the cluster so dashboards reflected real traffic. Security work focused on observation – packet capture and vulnerability scanning – and understanding, rather than claiming a complete hardened posture.

6. System Development Methodology

6.1 Methodology

The project used an incremental, iterative implementation methodology. Rather than designing the entire toolchain up front, each phase produced a working milestone that the next phase consumed – matching how DevOps capabilities are often adopted in real organizations, and keeping failure surfaces small enough to debug.

6.2 Project Phases

Phase	Focus	Primary outcome
0	Environment	Ubuntu Server VM, SSH, Git, Python, VS Code Remote-SSH, GitHub SSH key
1	Application	FastAPI CRUD, SQLite/SQLAlchemy, pytest, Docker image/container
2	CI	GitHub Actions workflow; Jenkins pipeline with real troubleshooting
3	Ansible	Control + target VMs, inventory, playbook, idempotency proof
4	Terraform	LocalStack S3 lifecycle; real AWS S3 with budget/IAM hygiene
5	Kubernetes	Minikube, Deployment (2 replicas), Service, ConfigMap, Secret
6	Helm	Chart with templates; install; raw manifests retained for comparison
7	Monitoring	Prometheus metrics, scraping, Grafana request rate and p95 panels
8	Security / network	tshark/Wireshark HTTP capture; Trivy image scan
9	Polish	Documentation, structure, portfolio readiness

6.3 Key Milestones

Milestone	Description	Evidence
M1	Remote Linux workflow operational	SSH session, Remote-SSH folder open
M2	API + DB + tests green	Swagger, pytest -v, SQLite file
M3	Container runs API	docker ps, /docs via published port
M4	CI green on push	GitHub Actions run; Jenkins SUCCESS
M5	Ansible idempotent	Changed count drops on second run
M6	Terraform apply/destroy	LocalStack and AWS S3 lifecycle
M7	App on Kubernetes	kubectl get pods, curl NodePort /health
M8	Helm release	helm install, curl still healthy
M9	Metrics dashboard	Grafana panels with live queries
M10	Capture + scan artifacts	pcapng in repo; Trivy report

6.4 Chronological Summary

Work began with hypervisor and Ubuntu Server installation, OpenSSH enablement, IP discovery, and SSH from Windows. System packages for Git and Python were installed; a virtual environment was verified; an Ed25519 key authenticated GitHub. TaskFlow was scaffolded, extended to SQLite-backed CRUD, tested, and containerized. CI was added in two forms. Ansible automated a second VM. Terraform practiced bucket lifecycle locally and on AWS. Minikube hosted a multi-replica Deployment with configuration injection. Helm packaged the same objects. Prometheus and Grafana observed traffic. Packet captures and Trivy scans closed the security-awareness loop. Final polish organized the repository for external readers.

7. System Testing and Evaluation

7.1 Testing Objectives and Methodology

Testing served two purposes: (1) verify TaskFlow behaves correctly as an API, and (2) verify each DevOps layer actually works (pipeline green, playbook idempotent, cluster healthy, metrics flowing). Methodology combined automated unit/API tests, manual exploratory checks via Swagger and curl, pipeline observations, and tool-specific verification commands (terraform plan/apply, kubectl get, helm status, Prometheus queries).

7.2 Test Environment

Primary environment: Ubuntu Server VM under VMware, Docker engine on the VM, Minikube using the Docker driver, Jenkins in Docker, LocalStack in Docker, and an optional AWS free-tier account. Browser-based checks used the host Windows machine reaching VM or Minikube published ports where networking allowed.

7.3 Test Cases

ID	Area	Action	Expected result
TC01	API	GET /health	JSON status ok
TC02	API	POST /tasks then GET /tasks	Created task listed
TC03	API	PUT/DELETE task by id	Updated or removed as specified
TC04	DB	Restart process, GET tasks	Data still present (SQLite)
TC05	Tests	pytest -v	Configured tests PASS
TC06	Docker	Build and run container	Health endpoint responds
TC07	GHA	Push to main	Workflow runs tests + build
TC08	Jenkins	Run pipeline	SUCCESS with tests + image
TC09	Ansible	Playbook twice	Second run minimal changes
TC10	Terraform	apply/destroy LocalStack	Bucket appears/disappears
TC11	Terraform	apply/destroy AWS	aws s3 ls reflects state
TC12	K8s	Deploy 2 replicas	Two pods Ready
TC13	K8s	Delete one pod	Replacement pod appears
TC14	K8s	curl NodePort /health	status ok
TC15	K8s	printenv in pod	ConfigMap/Secret values present
TC16	Helm	helm install	Release deployed, app healthy
TC17	Monitor	Prom targets / queries	TaskFlow targets UP; series non-empty
TC18	Monitor	Grafana panels	Rate and latency graphs render
TC19	Network	tshark capture + Wireshark	HTTP payload readable
TC20	Security	Trivy image scan	Report generated; findings reviewed

7.4 Bug Severity Classification

Severity	Meaning in this project
Critical	Blocks a phase milestone (e.g., cluster cannot run pods)
High	Major feature broken but workaround exists
Medium	Incorrect behaviour or flaky step requiring fix
Low	Cosmetic, documentation, or minor friction

Lifecycle used: New → Investigating → Fixed → Retested → Verified. Several issues required multiple investigation rounds, especially Jenkins environment completeness and Kubernetes image visibility.

7.5 Troubleshooting Case Studies

These cases are retained deliberately: operational competence is demonstrated as much by recovery as by first-time success.

Case 1 Python module typo (venc vs venv)

- What happened: `python3 -m venc` failed with No module named venc
- Cause: typographical error when creating a virtual environment
- Fix: re-ran `python3 -m venv` correctly; activated and deactivated successfully
- Lesson: CLI tools fail closed on typos; error text is part of the workflow

Case 2 FastAPI "Attribute app not found"

- What happened: Uvicorn reported the app attribute could not be found in the entrypoint module
- Cause: empty, unsaved, or misplaced main file; confusion between a single file and a package
- Fix: corrected file content and path; reload succeeded
- Lesson: ASGI entrypoints depend on exact module:variable paths

Case 3 Git upstream branch not set

- What happened: git push failed with "current branch has no upstream branch"
- Cause: remote repository existed but tracking was not wired on first push
- Fix: `git push --set-upstream origin main` after remote add
- Lesson: remotes and upstream tracking are explicit concepts, not automatic

Case 4 Jenkins missing Docker CLI despite socket access

- What happened: the pipeline could not execute docker build meaningfully
- Cause: socket access does not imply the docker binary is installed inside the container
- Fix: installed the Docker CLI inside the Jenkins environment; pipeline reached SUCCESS
- Lesson: self-hosted CI images must include every tool their stages invoke

Case 5 LocalStack S3 slow or hanging apply

- What happened: terraform apply against LocalStack hung or took excessive time
- Cause: S3 path-style addressing / provider endpoint configuration mismatch
- Fix: set `s3_use_path_style` and re-initialized; apply completed in seconds
- Lesson: local emulators need provider flags that real AWS may not require

Case 6 VM disk pressure

- What happened: filesystem space was exhausted during heavier Docker/LocalStack use
- Cause: accumulated Docker images, build cache, and logs over multiple phases
- Fix: docker system prune -a, apt autoremove, and journal vacuuming recovered headroom
- Lesson: even a learning cluster needs basic capacity management

Case 7 Kubernetes image not visible to Minikube

- What happened: pods failed to run the expected image built on the host Docker daemon
- Cause: Minikube runs its own separate Docker environment unless explicitly redirected
- Fix: used minikube docker-env and rebuilt the image inside the cluster's own context
- Lesson: where an image is built determines whether the cluster can see it without a registry

Case 8 Wrong NodePort / shell variable mistakes in curl

- What happened: a health check curl failed due to an assumed port and a shell expansion mistake
- Cause: the Service exposed a different NodePort than assumed
- Fix: read the actual port from kubectl get svc and corrected the command
- Lesson: service networking details must be read from cluster state, not assumed

8. Results and Discussion

8.1 Application Results

TaskFlow successfully exposed health and task CRUD endpoints, persisted data with SQLAlchemy and SQLite, and passed a small pytest suite. Interactive verification via Swagger UI and curl confirmed behaviour before automation layers were added.

8.2 Containerization Results

The Docker image built from the project Dockerfile ran the API on port 8000. Container start/stop/remove semantics were practiced deliberately to avoid name conflicts and to understand the difference between stopping and removing containers.

8.3 Continuous Integration Results

GitHub Actions executed checkout, Python setup, dependency install, tests, and image build with green runs on push. Jenkins achieved an equivalent SUCCESS status after environment gaps were fixed. The dual implementation highlighted trade-offs between managed and self-hosted CI.

8.4 Ansible Results

The playbook configured the target VM and demonstrated idempotency through reduced change counts on re-run. Connectivity was validated with an Ansible ping before applying longer playbooks.

8.5 Terraform and Cloud Results

LocalStack supported rapid S3 lifecycle practice. Real AWS work created and destroyed a bucket under a dedicated IAM user with budget alerting. No long-lived paid control planes were deployed; EKS was explicitly avoided due to continuous control-plane cost.

8.6 Kubernetes and Helm Results

Minikube ran two TaskFlow replicas behind a NodePort Service. Pod deletion demonstrated self-healing. ConfigMaps and Secrets injected environment values, verified with `kubectl exec printenv`. Helm installed a release from templated charts while raw manifests remained for comparison.

8.7 Monitoring Results

Prometheus scraped TaskFlow instances; queries such as request totals and rates returned series per pod. Grafana displayed request rate and filtered latency percentile panels suitable for demonstration at learning scale not a claim of production SRE maturity.

8.8 Security and Networking Results

Packet capture evidenced plaintext HTTP payloads on the lab path. Trivy produced a vulnerability report including dependency findings with available upgrades. These results support security discussion in interviews without overstating residual risk reduction.

8.9 Discussion

The project evolved from "an API process on a VM" into "an API whose build, configuration, infrastructure practice, deployment, and observation are expressed as code." The most important result is integration: each tool was forced to

consume outputs of earlier phases. Equally important is honesty about limits Minikube is not a multi-AZ production cluster; SQLite is not a managed database; free-tier experiments are not a full landing-zone design.

Component	Status
Linux VM + SSH + Remote-SSH	Implemented
GitHub-hosted application repo	Implemented
FastAPI + SQLite + pytest	Implemented
Docker image/container	Implemented
GitHub Actions CI	Implemented
Jenkins CI	Implemented (with troubleshooting)
Ansible configuration of target VM	Implemented
Terraform + LocalStack	Implemented
Terraform + AWS free-tier S3	Implemented (destroyed after test)
Minikube Deployment/Service/Config/Secret	Implemented
Helm chart install	Implemented
Prometheus + Grafana	Implemented at learning scale
tshark/Wireshark analysis	Implemented
Trivy scan	Implemented
Managed EKS / production HA	Not implemented (cost/scope)
Full automated production CD train	Not claimed
Managed SQL database	Not implemented

9. Conclusion and Future Work

9.1 Conclusion

This project set out to learn and demonstrate DevOps by building one coherent path around TaskFlow rather than collecting disconnected tool demos. That goal was met: a FastAPI service was developed on a remote Linux VM, tested, containerized, integrated through CI, configured with Ansible, supported by Terraform practice on LocalStack and AWS, deployed to Kubernetes, packaged with Helm, observed with Prometheus and Grafana, and examined with network capture and image scanning.

Automation improved the workflow by replacing repeated manual steps with version-controlled definitions and pipelines. The tools connected in a realistic order: tests before CI, images before orchestration, cluster before meaningful deployment automation, metrics after a stable service endpoint. Challenges from import path errors to Jenkins tooling gaps to Minikube image contexts were documented as part of the engineering record.

9.2 Limitations

- Portfolio/learning scale on a single primary VM and Minikube node
- SQLite rather than a production-grade managed database
- No managed Kubernetes control plane (EKS cost deliberately avoided)
- Limited cloud resource types beyond carefully scoped S3 exercises
- Absence of formal load testing and multi-region high availability
- Monitoring and security at demonstration depth, not full enterprise controls
- TLS termination and strict non-root policies discussed more than fully hardened

9.3 Future Work

- Deploying to a managed Kubernetes service with proper cost controls
- Replacing SQLite with managed PostgreSQL or similar
- GitOps workflows (e.g., Argo CD) for declarative cluster delivery
- Centralized logging and alerting rules beyond basic dashboards
- Stronger secrets management (e.g., external secret stores)
- Ingress with real TLS certificates
- Horizontal Pod Autoscaling under synthetic load
- Policy-as-code and tighter CI security gates
- Automated infrastructure teardown schedules and cost anomaly detection

These items are explicitly future work and are not presented as completed features of the current system.

References

- Docker Inc. Docker Documentation. <https://docs.docker.com/>
- Kubernetes Authors. Kubernetes Documentation. <https://kubernetes.io/docs/>
- Jenkins Project. Jenkins User Documentation. <https://www.jenkins.io/doc/>
- Red Hat Ansible. Ansible Documentation. <https://docs.ansible.com/>
- HashiCorp. Terraform Documentation. <https://developer.hashicorp.com/terraform/docs>
- LocalStack. LocalStack Documentation. <https://docs.localstack.cloud/>
- Amazon Web Services. AWS Documentation. <https://docs.aws.amazon.com/>
- Helm Authors. Helm Documentation. <https://helm.sh/docs/>
- Prometheus Authors. Prometheus Documentation. <https://prometheus.io/docs/>
- Grafana Labs. Grafana Documentation. <https://grafana.com/docs/>
- FastAPI Contributors. FastAPI Documentation. <https://fastapi.tiangolo.com/>
- Git Project. Git Documentation. <https://git-scm.com/doc>
- GitHub. GitHub Actions Documentation. <https://docs.github.com/actions>
- Aqua Security. Trivy Documentation. <https://aquasecurity.github.io/trivy/>
- Wireshark Foundation. Wireshark & tshark Documentation. <https://www.wireshark.org/docs/>
- OWASP Foundation. OWASP Top Ten and related application security guidance. <https://owasp.org/>
- Python Software Foundation. Python 3 Documentation. <https://docs.python.org/3/>
- SQLAlchemy Authors. SQLAlchemy Documentation. <https://docs.sqlalchemy.org/>

Appendices

Appendix A — Project Setup Summary

Install VMware Workstation Player; create an Ubuntu Server VM (2 vCPU, 4 GB RAM, ≥ 25 GB disk, NAT networking); enable OpenSSH during install; obtain the guest IP via `ip addr show`; SSH from Windows; run `sudo apt update && sudo apt upgrade -y`; install Git and Python tooling; configure git user.name/email; generate an Ed25519 key for GitHub; connect VS Code Remote-SSH to the VM and open the home directory.

Appendix B — FastAPI API Surface

Representative endpoints: GET /health, GET /, GET /tasks, POST /tasks, GET /tasks/{id}, PUT /tasks/{id}, DELETE /tasks/{id}. Interactive docs available at /docs when the server is running.

Appendix C — Docker Configuration Notes

Typical Dockerfile stages: base Python image; install requirements; copy application package; EXPOSE 8000; CMD Uvicorn on 0.0.0.0:8000. Rebuild after dependency upgrades identified by Trivy. Use distinct container names or remove old containers before re-creating to avoid name conflicts.

Appendix D — GitHub Actions Workflow (Conceptual)

Trigger on push to main; jobs run on ubuntu-latest; steps: checkout, setup-python, install requirements, pytest, docker build. Exact YAML lives in the repository under .github/workflows/.

Appendix E — Jenkinsfile (Conceptual)

Pipeline stages mirror CI: checkout, prepare Python environment, run pytest, docker build. Jenkins itself runs in Docker on the VM; tool installations inside the Jenkins environment were required to bring the pipeline to SUCCESS.

Appendix F — Ansible Inventory and Playbook

Inventory lists the target host(s). Playbook tasks included package cache update, Docker installation, service state, deployer user/group, and firewall rules. Success criterion: the second run shows idempotent behaviour (changed count drops).

Appendix G — Terraform Configuration

Separate roots/directories for LocalStack versus real AWS help avoid accidental cloud changes. Provider endpoints and path-style settings matter for LocalStack. State and lock files are gitignored. Always plan before apply; destroy when experiments end.

Appendix H — Kubernetes YAML Files

Repository paths under kubernetes/ include Deployment (replicas: 2), Service (NodePort), ConfigMap, and Secret manifests. Verify with `kubectl get pods, svc` and `curl` to the node IP and nodePort.

Appendix I — Helm Chart Structure

Chart layout: Chart.yaml, values.yaml, templates/ for Deployment, Service, ConfigMap. helm template before helm install.

Appendix J — Prometheus Configuration Notes

Application exposes metrics; Service annotations enable discovery; confirm targets UP; use rate and histogram quantile queries carefully filtered by service labels.

Appendix K — Grafana Notes

Add Prometheus as a data source; create dashboard panels for request rate and p95 latency with label filters so unrelated metrics do not pollute graphs.

Appendix L — Security Scanning

Trivy image scan output is archived as a project artifact, distinguishing base image OS noise from application dependency findings that can be upgraded directly.

Appendix M — Final Directory Structure (Representative)

backend/ (app, tests, requirements, Dockerfile), .github/workflows/, Jenkinsfile, ansible/, terraform/, terraform-aws/, kubernetes/, helm/ (or charts directory), monitoring notes, docs/ (including capture artifacts), README.

This document records a portfolio-scale implementation intended to demonstrate applied DevOps competency across the full toolchain. It is written for technical review by hiring teams and engineering peers.